

Software Engineering Masterclass

Principles, Process Models, Design, Testing & Project Management

Presented by: Asad Usman

Module 1: Introduction to Software Engineering

1. What is Software Engineering?

- **Definition:** The systematic, disciplined, and quantifiable application of engineering principles to the design, development, operation, and maintenance of software.
- **Software Crisis:** Historical term addressing issues of delayed projects, budget overruns, poor quality, and unmaintainable code.
- **Attributes of Good Software:** Maintainability, dependability, security, efficiency, and acceptable usability.

Module 2: Software Process Models

1. Traditional vs. Agile Models

- **Waterfall Model:** Linear sequential approach (Requirement Analysis → Design → Implementation → Testing → Deployment). Rigid and inflexible to changing requirements.
- **Agile & Scrum:** Iterative and incremental development focusing on adaptive planning, rapid delivery, customer collaboration, and cross-functional teams.
- **Spiral Model:** Risk-driven process model combining iterative development with systematic waterfall aspects.

Module 3: Requirements Engineering

1. The Requirements Process

- **Elicitation & Analysis:** Gathering needs from stakeholders through interviews, surveys, and use-case analysis.
- **Specification:** Documenting requirements formally into an SRS (Software Requirements Specification) document.
- **Functional vs. Non-Functional:** Functional requirements define system features and behaviors; non-functional requirements govern performance, security, and scalability.

Module 4: Software Design & Architecture

1. Core Design Principles

- **Modularity:** Dividing a complex system into manageable, independent modules.
- **Coupling & Cohesion:** Aiming for *high cohesion* (modules perform focused tasks) and *low coupling* (minimal dependencies between modules).
- **Design Patterns:** Reusable solutions to commonly occurring software design problems (e.g., Creational, Structural, Behavioral patterns).
- **Architectural Patterns:** High-level structural schemes like Model-View-Controller (MVC) and Microservices.

Module 5: Software Testing & Quality Assurance

Verification and validation processes ensuring software meets requirements and operates without critical defects.

Testing Level / Type	Description & Purpose
Unit Testing	Testing individual functions, classes, or components in isolation (typically written by developers).
Integration Testing	Verifying interactions and data flow between integrated modules and subsystems.
System Testing	Evaluating the fully integrated software product against specified functional requirements.
Acceptance Testing	Validation performed by the client or end-users to confirm the system meets business needs.
Black-Box vs. White-Box	Black-box tests functionality without internal code knowledge; white-box tests internal logic paths.

Module 6: Maintenance & Project Management

1. Software Maintenance Types

- **Corrective Maintenance:** Fixing bugs and operational defects discovered post-release.
- **Adaptive Maintenance:** Updating software to work in evolving operating environments or hardware.
- **Perfective Maintenance:** Adding new features and enhancing performance or maintainability.

2. Project Management & Estimation

- **Estimation Techniques:** Algorithmic models like COCOMO (Constructive Cost Model) and expert judgment.
- **Risk Management:** Identifying project risks (schedule slips, resource turnover, technical debt) and formulating mitigation strategies.

Summary Note: Mastery of software engineering principles ensures sustainable, high-quality, and scalable software systems capable of evolving with user needs.